Image Segmentation

Avinash Sharma

CSE Department, IIT Jodhpur

Image courtesy: https://www.etsy.com/in-en/listing/1607436850/third-eye-activation-spiritual-awakening

Slides from Fei-Fei Li's lectures

Computer Vision Tasks

Classification

CAT

No spatial extent

Semantic Segmentation

Object Detection

Instance Segmentation

Semantic Segmentation

Classification

Semantic Segmentation

Object Detection

Instance Segmentation

CAT

No spatial extent

Semantic Segmentation: The Problem

GRASS, **CAT**, **TREE**, **SKY**, **...**

Paired training data: for each training image, each pixel is labeled with a semantic category.

?

At test time, classify each pixel of a new image.

Semantic Segmentation Idea: Sliding Window

Full image

?

Semantic Segmentation Idea: Sliding Window

Full image

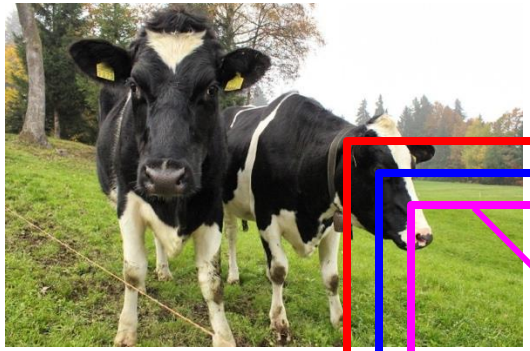
?

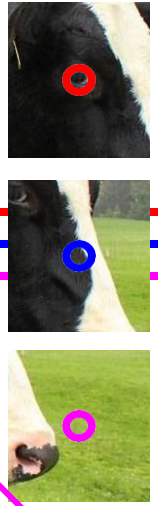
Impossible to classify without context

Q: how do we include context?

Semantic Segmentation Idea: Sliding Window

Full image

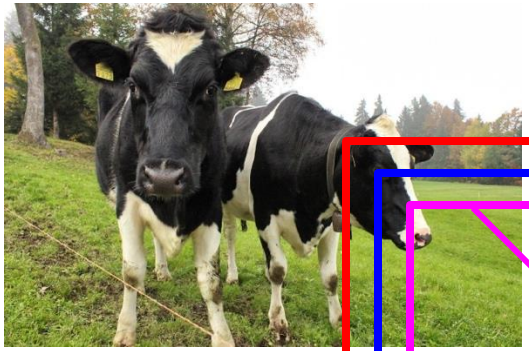




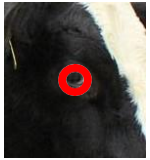
Q: how do we model this?

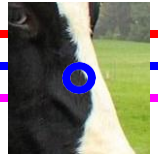
Semantic Segmentation Idea: Sliding Window

Full image

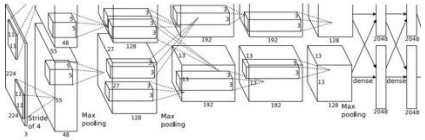


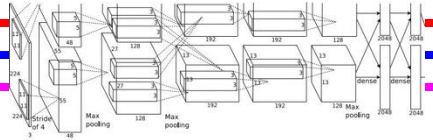
Extract patch

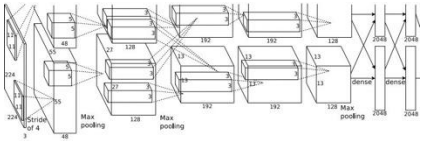




Classify center pixel with CNN







Cow

Cow

Grass

Farabet et al, “Learning Hierarchical Features for Scene Labeling,” TPAMI 2013

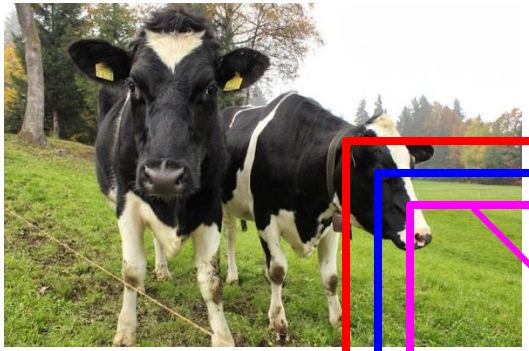
Pinheiro and Collobert, “Recurrent Convolutional Neural Networks for Scene Labeling”, ICML 2014

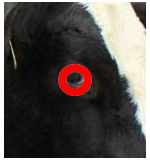
Semantic Segmentation Idea: Sliding Window

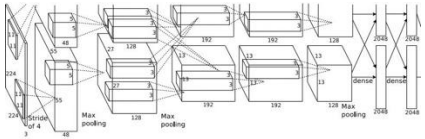
Extract patch

Classify center pixel with CNN

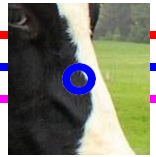
Full image

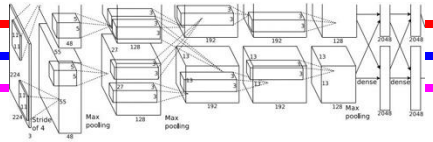




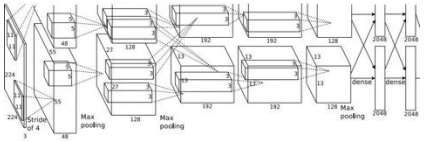


Cow





Cow



Grass

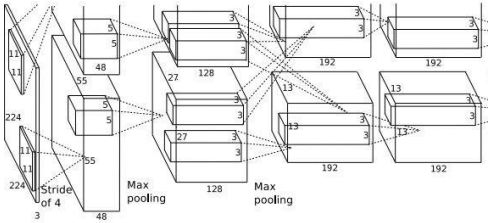
Problem: Very inefficient. Not reusing shared features between overlapping patches

Farabet et al, “Learning Hierarchical Features for Scene Labeling,” TPAMI 2013

Pinheiro and Collobert, “Recurrent Convolutional Neural Networks for Scene Labeling”, ICML 2014

Semantic Segmentation Idea: Convolution

Full image



An intuitive idea: encode the entire image with conv net, and do semantic segmentation on top.

Problem: classification architectures often reduce feature spatial sizes to go deeper, but semantic segmentation requires the output size to be the same as input size.

Semantic Segmentation Idea: Fully Convolutional

Design a network with only convolutional layers without downsampling operators to make predictions for pixels all at once!

Input:
3 x H x W

Conv → Conv → Conv → Conv → argmax

Convolutions:
D x H x W

Scores:
C x H x W

Predictions:
H x W

Semantic Segmentation Idea: Fully Convolutional

Design a network with only convolutional layers without downsampling operators to make predictions for pixels all at once!

Input:
3 x H x W

Conv → Conv → Conv → Conv → argmax

Convolutions:
D x H x W

Scores:
C x H x W

Predictions:
H x W

Problem: convolutions at original image resolution will be very expensive ...

Semantic Segmentation Idea: Fully Convolutional

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!

Input:
3 x H x W

High-res:
D_1 x H/2 x W/2

Med-res:
D_2 x H/4 x W/4

Low-res:
D_3 x H/4 x W/4

Med-res:
D_2 x H/4 x W/4

High-res:
D_1 x H/2 x W/2

C x H x W

Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015

Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

Semantic Segmentation Idea: Fully Convolutional

Downsampling: Pooling, strided convolution

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!

Upsampling: ???

Input:
3 x H x W

High-res:
D_1 x H/2 x W/2

Med-res:
D_2 x H/4 x W/4

Low-res:
D_3 x H/4 x W/4

Med-res:
D_2 x H/4 x W/4

High-res:
D_1 x H/2 x W/2

C x H x W

Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015

Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

In-Network upsampling: “Unpooling”

Nearest Neighbor

1	2
3	4

Input: 2 x 2

1	1	2	2
1	1	2	2
[illegible]	[illegible]	4	4

“Bed of Nails”

1	2
3	4

Input: 2 x 2

1	0	2	0
0	0	0	0
[illegible]	0	4	0

In-Network upsampling: “Max Unpooling”

Max Pooling
Remember which element was max!

1	2	6	3
3	5	2	1
1	2	2	1
7	3	4	8

Input: 4 x 4

5	6
7	8

Output: 2 x 2

…

Rest of the network

Max Unpooling
Use positions from pooling layer

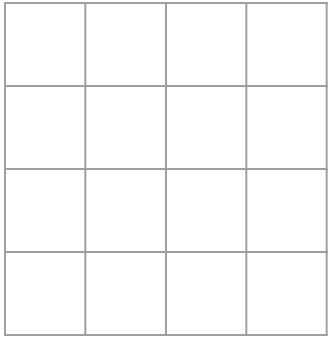
1	2
3	4

0	0	2	0
0	1	0	0
	0	0	0

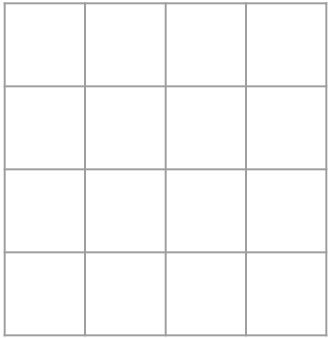
Corresponding pairs of downsampling and upsampling layers

Learnable Upsampling

Recall: Normal 3 x 3 convolution, stride 1 pad 1



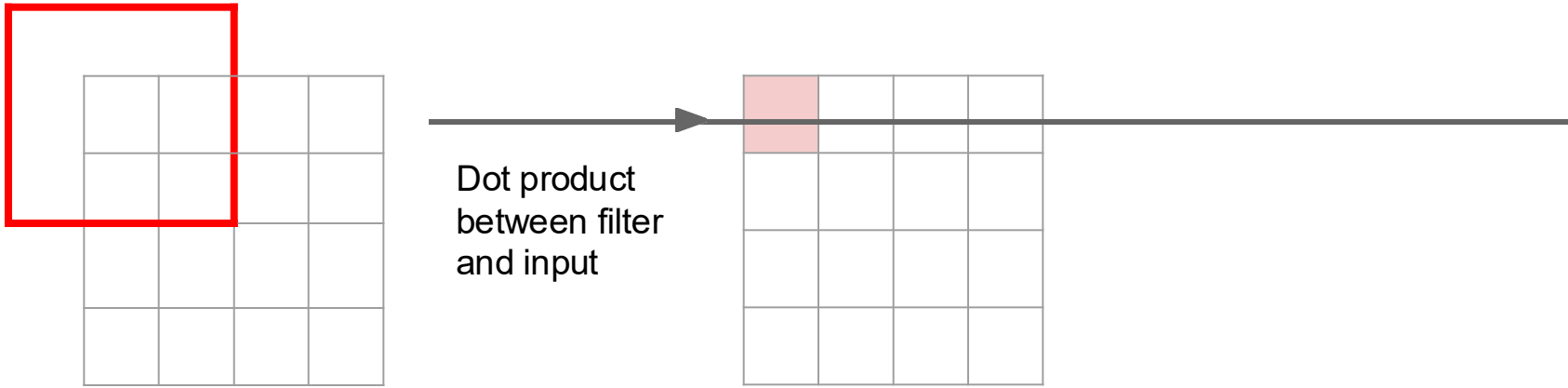
Input: 4 x 4



Output: 4 x 4

Learnable Upsampling

Recall: Normal 3 x 3 convolution, stride 1 pad 1

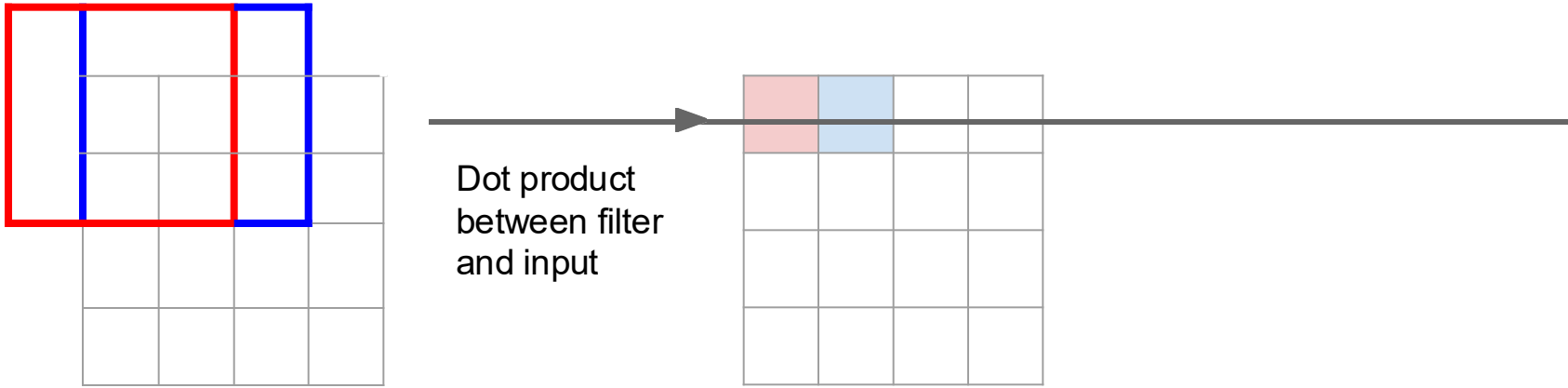


Input: 4 x 4

Output: 4 x 4

Learnable Upsampling

Recall: Normal 3 x 3 convolution, stride 1 pad 1

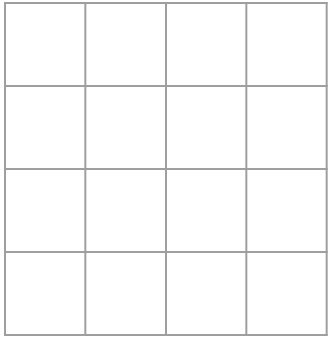


Input: 4 x 4

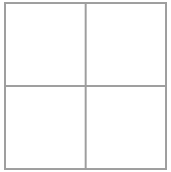
Output: 4 x 4

Learnable Upsampling

Recall: Normal 3 x 3 convolution, stride 2 pad 1



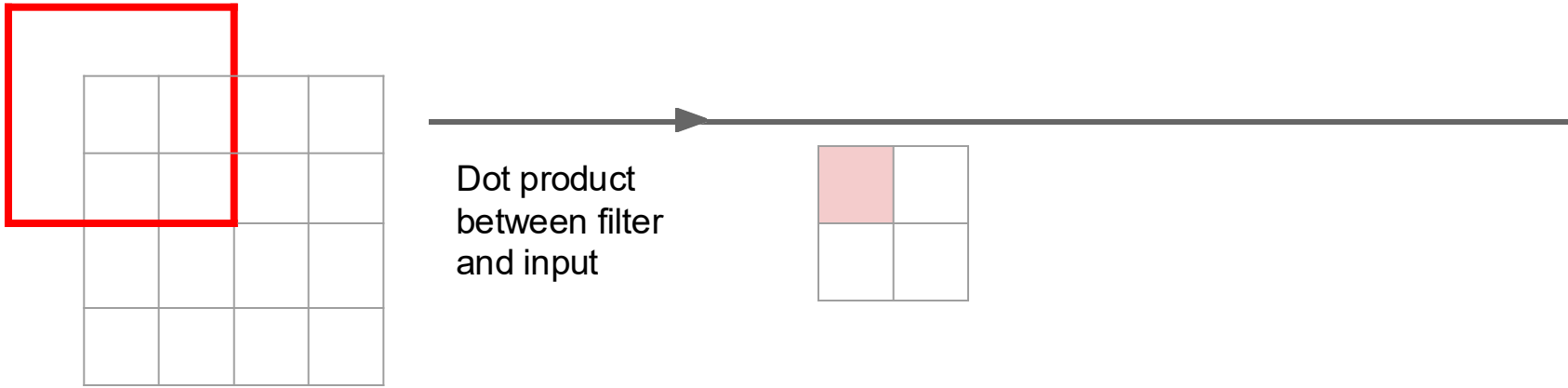
Input: 4 x 4



Output: 2 x 2

Learnable Upsampling

Recall: Normal 3 x 3 convolution, stride 2 pad 1

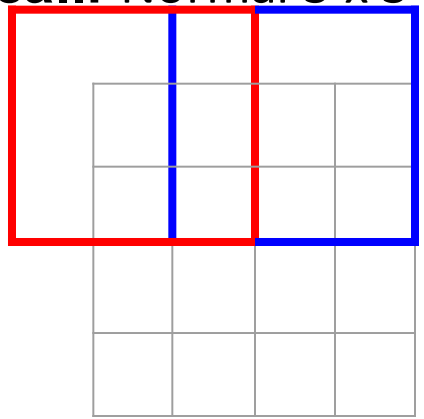


Input: 4 x 4

Output: 2 x 2

Learnable Upsampling

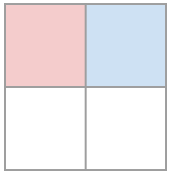
- **Recall:** Normal 3 x 3 convolution, stride 2 pad 1



Input: 4 x 4

Dot product between filter and input

Filter moves 2 pixels in the input for every one pixel in the output



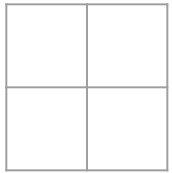
Output: 2 x 2

Stride gives ratio between movement in input and output

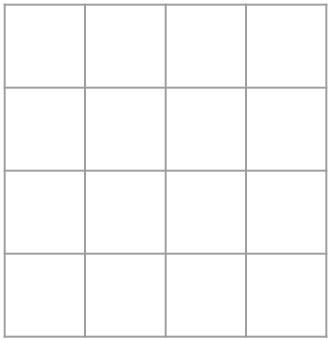
We can interpret strided convolution as “learnable downsampling”.

Learnable Upsampling: Transposed Convolution

3 x 3 **transposed** convolution, stride 2 pad 1



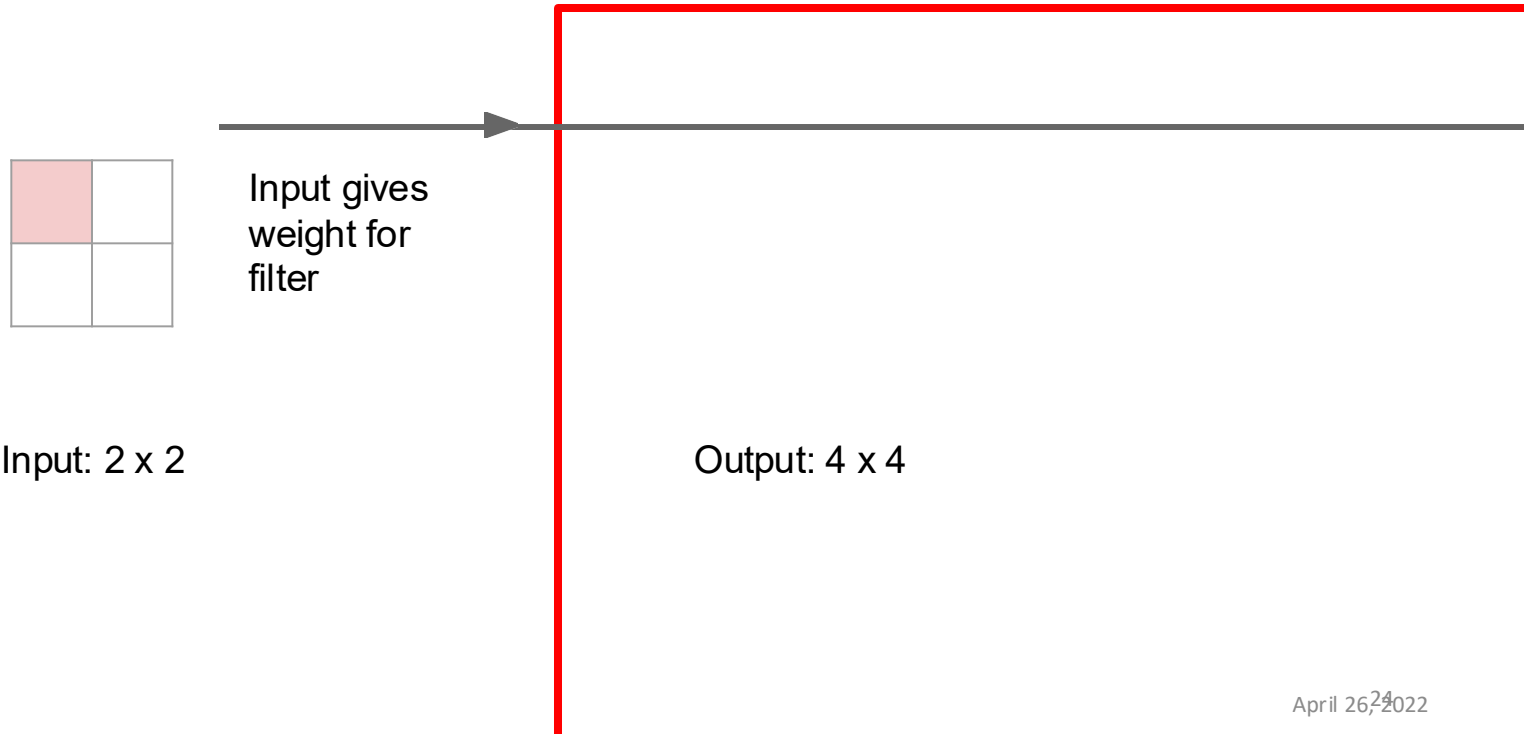
Input: 2 x 2



Output: 4 x 4

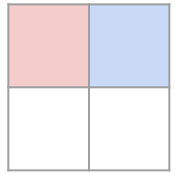
Learnable Upsampling: Transposed Convolution

3 x 3 **transposed** convolution, stride 2 pad 1



Learnable Upsampling: Transposed Convolution

3 x 3 **transposed** convolution, stride 2 pad 1



Input gives weight for filter

Filter moves 2 pixels in the output for every one pixel in the input

Stride gives ratio between movement in output and input

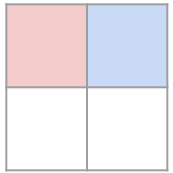
Input: 2 x 2

Output: 4 x 4

Learnable Upsampling: Transposed Convolution

3 x 3 **transposed** convolution, stride 2 pad 1

Sum where output overlaps



Input gives weight for filter

Filter moves 2 pixels in the output for every one pixel in the input

Stride gives ratio between movement in output and input

Input: 2 x 2

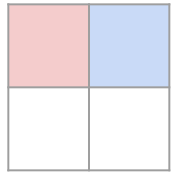
Output: 4 x 4

Learnable Upsampling: Transposed Convolution

Q: Why is it called transposed convolution?

3 x 3 **transposed** convolution, stride 2 pad 1

Sum where output overlaps



Input gives weight for filter

Filter moves 2 pixels in the output for every one pixel in the input

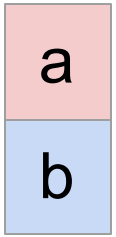
Stride gives ratio between movement in output and input

Input: 2 x 2

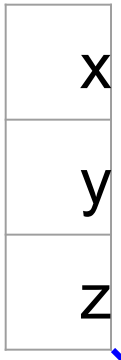
Output: 4 x 4

Learnable Upsampling: 1D Example

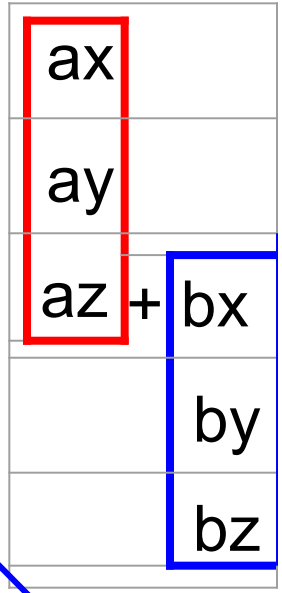
Input



Filter



Output



Output contains copies of the filter weighted by the input, summing at where at overlaps in the output

Convolution as Matrix Multiplication (1D Example)

We can express convolution in terms of a matrix multiplication

$$\vec{x} * \vec{a} = X\vec{a}$$

$$\begin{bmatrix} x & y & z & 0 & 0 & 0 \\ 0 & 0 & x & y & z & 0 \end{bmatrix} \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ bx + cy + dz \end{bmatrix}$$

Example: 1D conv, kernel size=3, stride=2, padding=1

Convolution as Matrix Multiplication (1D Example)

We can express convolution in terms of a matrix multiplication

$$\vec{x} * \vec{a} = X\vec{a}$$

$$\begin{bmatrix} x & y & z & 0 & 0 & 0 \\ 0 & 0 & x & y & z & 0 \end{bmatrix} \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ bx + cy + dz \end{bmatrix}$$

Example: 1D conv, kernel size=3, stride=2, padding=1

Transposed convolution multiplies by the transpose of the same matrix:

$$\vec{x} *^T \vec{a} = X^T\vec{a}$$

$$\begin{bmatrix} x & 0 \\ y & 0 \\ z & x \\ 0 & y \\ 0 & z \\ 0 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} ax \\ ay \\ az + bx \\ by \\ bz \\ 0 \end{bmatrix}$$

Example: 1D transposed conv, kernel size=3, stride=2, padding=0

Semantic Segmentation Idea: Fully Convolutional

Downsampling: Pooling, strided convolution

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!

Upsampling: Unpooling or strided transposed convolution

Input:
3 x H x W

High-res:
D_1 x H/2 x W/2

Med-res:
D_2 x H/4 x W/4

Low-res:
D_3 x H/4 x W/4

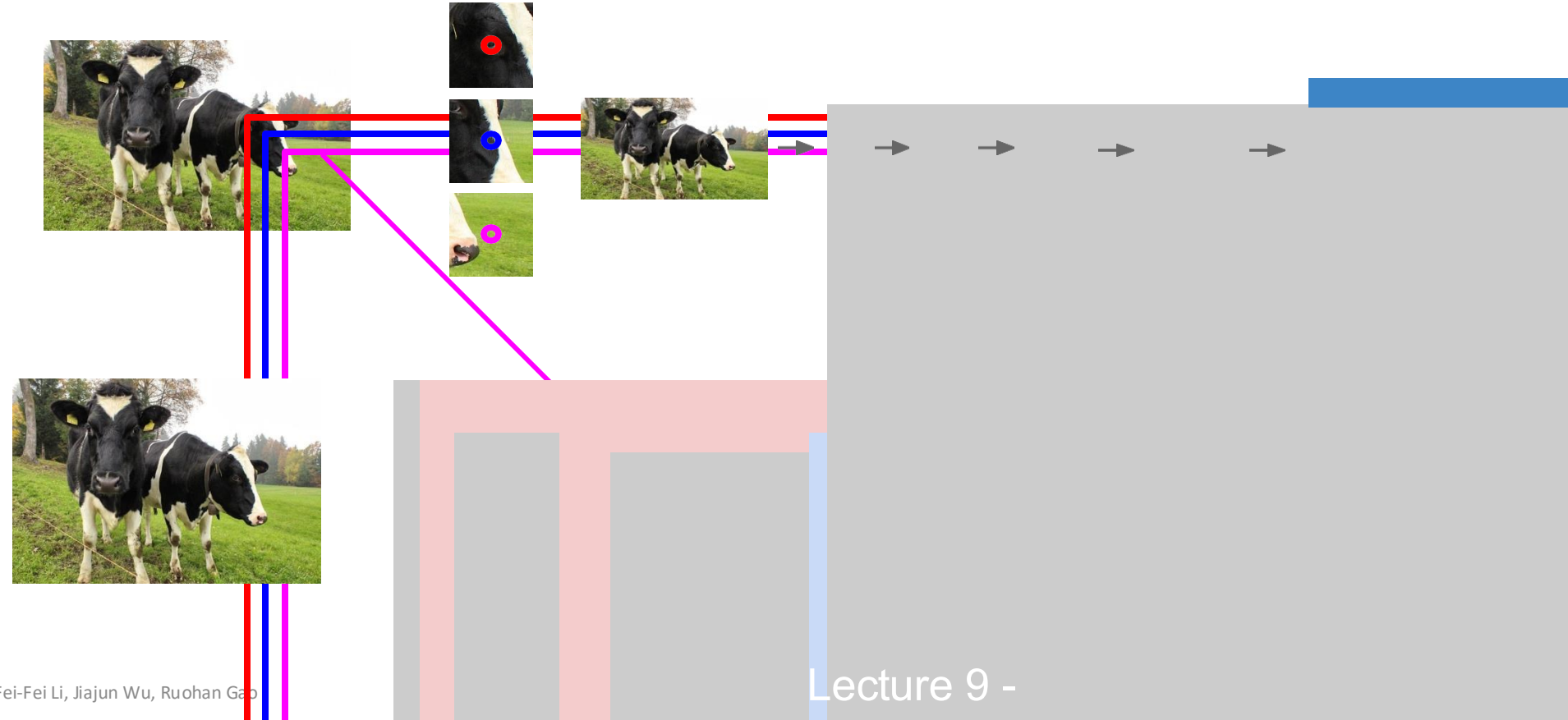
Med-res:
D_2 x H/4 x W/4

High-res:
D_1 x H/2 x W/2

Predictions:
H x W

Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015 Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

Semantic Segmentation: Summary



U-Net

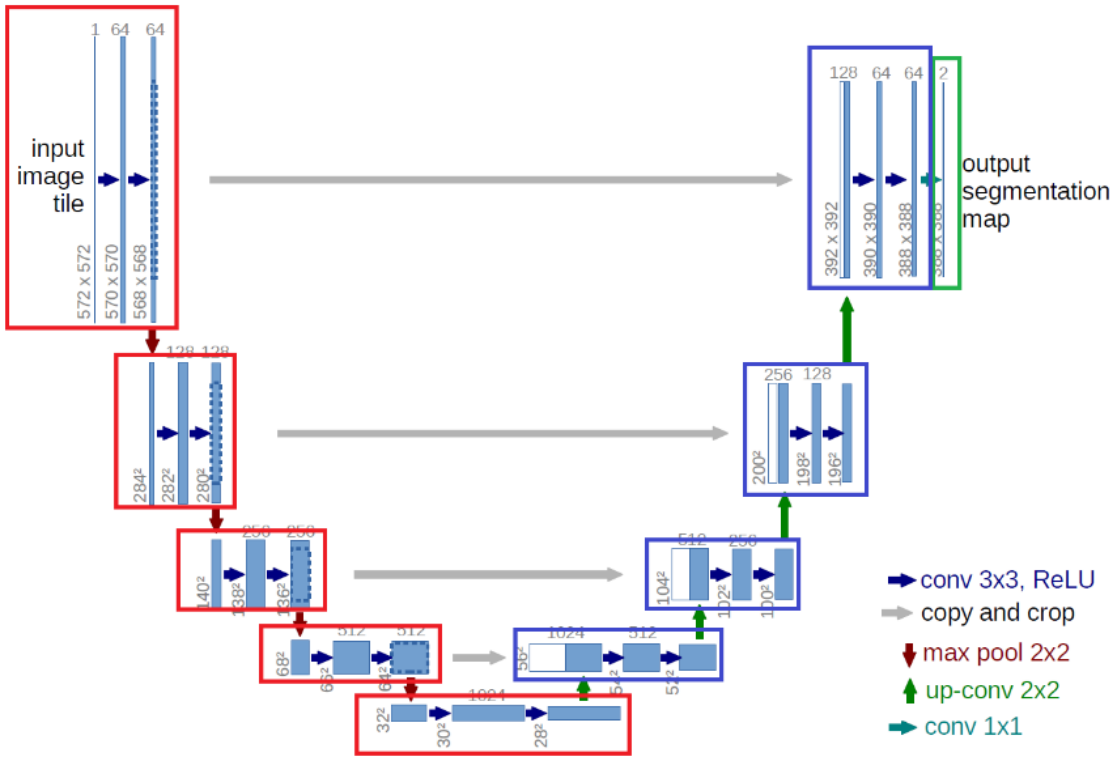


Fig. 1. U-net architecture (example for 32x32 pixels in the lowest resolution). Each blue box corresponds to a multi-channel feature map. The number of channels is denoted on top of the box. The x-y-size is provided at the lower left edge of the box. White boxes represent copied feature maps. The arrows denote the different operations.

Semantic Segmentation

Label each pixel in the image with a category label

Don’t differentiate instances, only care about pixels

This image is CC0 public domain